

AI提示语工程

软件开发人员专属实战课程



目录



01. 开发者视角：核心价值

理解 AI 提示语工程在开发中的核心价值与定位



02. 核心原则：底层逻辑

掌握开发场景下设计优质提示语的八大底层原则



03. 万能模板：高性价比设计

学习应对 80% 开发场景的高性价比提示语通用模板



04. 场景化技巧：流程优化

结合具体开发场景，分享全流程的提示语优化策略



05. 高级技巧：释放高阶能力

探索提示语工程的高级玩法，释放 AI 的深层潜力



06. 实战演练与误区规避

通过实战案例巩固知识，识别并规避常见的开发误区

01

开发者视角：AI提示语工程的核心价值

为什么开发者必须掌握提示语工程？

什么是AI提示语工程？

核心定义：系统性方法

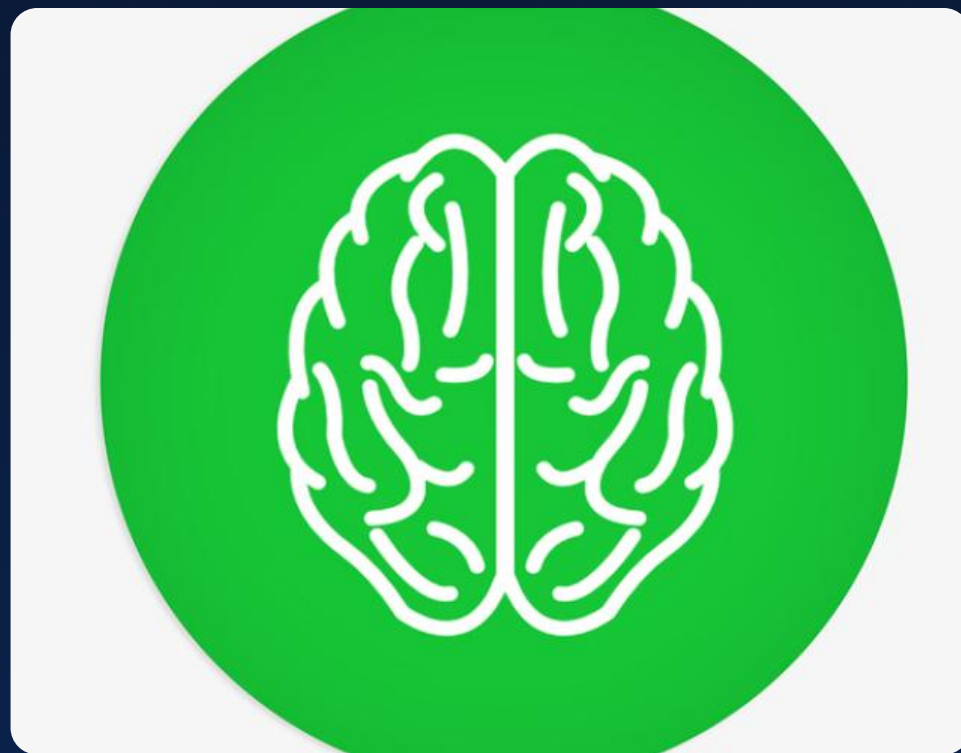
通过设计结构化、精准化、场景化的文本指令，引导AI模型生成符合预期的结果。

适用场景：主流模型

广泛应用于 ChatGPT、文心一言、CodeLlama 等各类生成式 AI 模型。

开发者价值：需求转化

将业务需求转化为 AI 可执行的开发任务，是连接人类意图与机器能力的关键桥梁。



为什么开发者必须掌握？

糟糕的提示词导致 AI 生成质量低下，效率反而降低



生成的代码语法正确但不符合业务逻辑，无法直接使用



调试建议泛泛而谈，无法定位具体 Bug，浪费排查时间



接口文档格式混乱，不符合公司规范，需二次整理



测试用例覆盖不全，遗漏核心场景，存在潜在风险



技术方案缺乏落地性，与现有架构冲突，难以集成

为什么开发者必须掌握？（价值）



代码生成

一次产出符合规范的可运行代码，避免反复调试语法错误。



Bug调试

精准定位问题根因，并直接给出符合逻辑的修复方案。



文档编写

直接生成符合公司规范的接口文档，保持文档与代码同步。



测试设计

快速生成覆盖核心场景的标准化测试用例，提升测试覆盖率。



方案设计

基于现有架构，快速输出可落地的技术方案原型。



核心收益

减少**80%**的AI输出返工成本，实现开发效率质的飞跃。

02

核心原则：开发场景优质提示语的底层逻辑

掌握这8大原则，让AI成为你的得力助手。

原则1：具体 - 拒绝模糊，定义到“可执行”

避免使用模糊的指令，将需求具体化到技术栈、业务场景、功能要求和输出格式，确保指令清晰可落地。

✖ 反面教材：模糊的指令

“写一个Python接口”

问题分析：

- 未指定框架（Flask/Django/FastAPI?）
- 未说明业务场景（登录/支付/查询?）
- 未定义输入输出参数格式

✔ 正面示范：可执行的指令

- 技术栈：Python 3.9 + FastAPI
- 业务场景：用户登录鉴权接口
- 功能要求：接收POST参数(username/password)，验证后返回JWT Token
- 输出格式：完整代码 + requirements.txt + 接口测试示例

原则2：角色设定 - 给AI明确的“开发身份”



核心逻辑：为AI设定具体的专家角色（如资深工程师）并明确沟通对象，引导AI输出更专业、更贴合场景的深度回答。



糟糕示例：指令模糊

“解释一下分布式锁”

问题分析：缺乏上下文，AI无法判断回答的深度（是给新人科普还是给架构师讨论），容易得到过于笼统或基础的百科式回答。



优质示例：角色赋能

“请你扮演一位有5年分布式系统开发经验的Java后端工程师，向刚接触分布式的初级开发解释Redis分布式锁的实现原理...”

优势分析：通过设定“资深Java后端工程师”角色和“初级开发”受众，AI能精准控制专业术语的使用，并提供如Redis代码等实战内容。

原则3：上下文补充 - 提供开发环境/架构约束

提供完整的上下文信息（如运行环境、架构约束、性能要求），是AI生成有效代码的前提。

✖ 糟糕示例：信息缺失

用户指令：

“帮我优化这段代码。”

问题分析：

- 未说明使用的语言版本（如Python 2还是3）
- 未提及项目框架（如Flask/Django）
- 缺乏性能指标（如QPS要求）和架构限制

✔ 优质示例：上下文完备

用户指令：

- [环境] Python 3.8, Flask框架
- [约束] 必须使用连接池，禁止直连数据库
- [性能] 支持1000QPS，需添加分页功能
- [指令] 优化以下Python代码，提升性能和可维护性。

原则4：输出格式指定 - 符合开发规范的格式要求

明确指定输出格式（如Markdown表格、代码结构、文档模板），能显著提升AI输出的可用性。

✗ 糟糕示例：指令模糊

“写接口文档”

问题：缺乏具体标准，AI可能返回非结构化的文字描述，难以直接用于开发。

✓ 优质示例：指令清晰

- **【文档规范】** 符合公司 OpenAPI 3.0 规范
- **【接口信息】** 用户信息查询 (GET /api/v1/user/{id})
- **【输出要求】** 按指定 Markdown 格式生成，包含基本信息、请求参数、响应结果和错误码说明。

03



万能模板：开发场景高性价比提示语设计

一个模板，应对80%的开发场景。

开发者专属核心模板



01 开发角色设定

明确你希望扮演的专家身份，如资深架构师、前端开发等。



02 环境与架构约束

指定技术栈（如React, Python）、运行环境及架构限制。



03 业务功能需求

详细描述要实现的具体功能逻辑、流程及交互细节。



04 技术规范要求

定义代码风格、注释规范、性能指标及安全标准。



05 输出与验收标准

明确最终交付物的形式（如API文档、代码包）及验收标准。



06 参考示例 (可选)

针对复杂任务，提供输入输出示例以确保理解一致性。



通用框架提示：这是一个标准化的组合结构，可根据具体任务的复杂程度灵活增删模块。

模板实战：后端开发示例

基础开发环境

开发角色

Java后端工程师

技术环境

SpringBoot 2.7 + MySQL 8.0 +
MyBatis-Plus

架构约束

微服务架构，接入统一异常处理
与日志框架

业务与技术规范

核心业务功能

实现商品库存扣减接口，包含库存原子性扣减与操作日志记录。

技术规范要求

- 遵循阿里巴巴Java开发手册
- 严格控制事务边界
- 实现防超卖机制

输出交付标准

完整代码实现

Controller + Service + Mapper 三层架构完整代码，附详细注释。

单元测试覆盖

提供基于 JUnit5 的单元测试用例，覆盖正常与异常场景。

接口测试示例

包含 HTTP 请求参数与响应结果示例。

04

场景化技巧：开发全流程提示语优化

针对代码生成、文档编写、测试用例等具体场景的优化技巧。



DEVELOPMENT WORKFLOW

场景1：代码生成与调试

✖ 常见误区：指令模糊

Prompt:

“帮我调试这段代码，看看哪里有问题？”

问题分析：

- 缺乏上下文：未说明业务场景与运行环境
- 现象描述模糊：未提及高并发等关键条件
- 目标不明确：未指定修复后的预期结果

✔ 专家范式：结构化指令

👤 角色设定：资深Java后端工程师 (微服务领域)

💻 技术环境：SpringBoot 2.7 + MyBatis + RabbitMQ

🐛 Bug现象：高并发下部分订单状态更新后消息未发送，导致下游服务数据不一致。

⚡ 执行指令：

- 分析代码中的问题根因 (包含事务、消息发送时机)
- 提供修复后的完整代码 (需保证事务一致性)
- 说明分布式事务解决方案

05

高级技巧：释放AI的开发高阶能力

掌握少样本学习、思维链等高级方法，解决更复杂的开发问题。



技巧2：思维链(CoT) - 引导AI分步解决复杂问题



普通提示 (Normal Prompt)

“设计一个高并发的订单系统”

特点：指令简单直接，缺乏步骤引导。AI往往只能给出笼统、碎片化的答案，难以覆盖复杂的业务场景和技术细节。



思维链提示 (Chain of Thought)

“请分步设计高并发订单系统(日单量100万+)，按以下步骤思考：”

- 步骤1：分析核心痛点 (性能/一致性/可用性)
- 步骤2：设计核心架构 (分层/分库分表/缓存)
- 步骤3：选型核心技术栈 (语言/框架/中间件)
- 步骤4：设计核心流程 (下单/支付/库存扣减)
- 步骤5：性能优化与容灾方案
- 步骤6：提供核心代码示例 (如库存扣减逻辑)

06

实战演练与开发场景误区规避

避坑指南与自检清单

开发场景常见误区与自检清单

常见提示语误区 (必避)

-  只提功能不提技术栈/版本
-  忽略编码规范与代码风格
-  不提架构约束与依赖关系
-  只关注功能不关注性能/安全
-  要求AI生成“可直接上线”的代码
-  粘贴大段无标注代码，缺乏上下文

开发提示语自检清单

-  明确技术栈及具体版本号
-  明确编码规范（如命名、注释要求）
-  明确架构约束与接口定义
-  明确性能指标与安全合规要求
-  明确输出格式（JSON/XML/代码块）
-  提供充分的上下文信息与业务背景

课程总结



核心原则与模板

掌握AI提示语工程的核心原则，灵活运用万能模板，是驾驭AI的基础。



场景化与高级技巧

结合具体开发场景，运用高级提示语技巧，显著提升开发效率与代码质量。



感谢观看